Technical report: Nearby Stars, Plain Words

- **Track:** Track 2B, Own Project
- **Event:** Hack Apertus, online
- **Team:** rC (solo), Ram Charan Polisetti
- **Demo:** video URL to be added on the submission form
- **Repository:** https://github.com/ram-polisetti/apertus-nearby-stars

1. Summary

Beginners meet astronomy through questions, not catalogues: "What is the closest star?", "How far is Vega?", "How many stars are near us?" General chatbots answer such questions from memory and sometimes invent numbers. Nearby Stars, Plain Words answers them from a real catalogue instead. The app holds the 1,113 nearest stars from ESA's Gaia Data Release 3. For each question, a small retrieval layer first extracts the relevant facts from the data. Apertus then phrases those facts in plain language under a strict instruction to use only the facts given. If no model endpoint is configured, or the call fails, a deterministic fallback answers from the same facts, so the app is fully functional offline. The headline result: every factual claim the app makes can be traced to a row of the Gaia extract, whether or not the language model is in the loop.

2. Architecture

The whole system is one Python 3 file (`track_2b/src/app.py`), standard library only, plus a single-page web UI. Components:

1. **Dataset (bundled).** `data/nearby_stars.csv`: 1,113 stars with position, parallax, distance, Gaia G magnitude, BP-RP colour and GSP-Phot surface temperature where available.
2. **Retrieval.** Pattern-based intent matching over the question: closest stars, brightest stars, hottest stars, count within a given distance (light years or parsecs), and lookup of a named star. It returns a facts text plus a ready-made plain answer built from the same facts.
3. **Phrasing (optional).** If `LLM_BASE_URL` is set, the facts and the question are sent to an OpenAI-compatible Apertus endpoint. The response replaces the wording only; the JSON reply also carries the raw `facts` and a `source` field (`apertus` or `data-fallback`) so any answer can be audited.
4. **HTTP layer.** `GET /` serves the UI, `POST /ask` takes `{"question": ...}`, `GET /health` reports dataset size and whether an LLM is configured.

The container (`track_2b/Dockerfile`, `python:3.12-slim`) installs nothing: there are no third-party dependencies, so the build cannot fail on package downloads and the image stays small. `make run` at the repository root (or in `track_2b/`) builds and starts it on port 8000.

Target architecture (mandatory)

This project is deployable in architecture **a) on-premise** and **b) air-gapped**, and the two collapse into the same setup:

- The app, the dataset and (optionally) the model all run on the operator's own infrastructure. At runtime the app makes exactly one kind of outbound call: to the Apertus endpoint named by `LLM_BASE_URL`. Point that variable at an Apertus deployment served on the same machine or the same local network (for example the open Apertus weights served with vLLM or SGLang) and nothing leaves the site.

- With the LLM variables unset, the app needs no network access at all: questions are answered from the bundled CSV by the deterministic fallback. This is a true air-gapped mode, useful for classrooms, planetariums and field sites.
- Build time versus runtime: the only network use at build time is pulling the public `python:3.12-slim` base image. There are no package installs. The Gaia data ships inside the repository, so no data download happens at build or run time.
- The same container also fits architecture c) unchanged: it can be deployed on a sovereign Swiss cloud with a Swiss-hosted Apertus endpoint, but a) and b) are the primary, tested targets.

3. Use of Apertus

- **Model:** swiss-ai/Apertus-v1.5-8B (default; any model name can be set with `LLM_NAME`)
- **How it is used:** inference only, as a constrained phrasing layer
- **Where it runs:** wherever the operator points `LLM_BASE_URL`; an OpenAI-compatible `/chat/completions` endpoint. The app itself is model-agnostic and was tested against a local mock endpoint with the same interface.

Configuration is only through the environment, as the template requires: `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY`. The system prompt is short and strict: answer using only the facts provided, plain words, short sentences, no jargon, no em dashes, and if the facts do not cover the question, say what they do cover instead of guessing. Temperature is 0.3 and replies are capped at 220 tokens. Any error, timeout (30 s) or missing configuration silently falls back to the deterministic answer; the `source` field in the response records which path answered.

The design decision worth defending: Apertus is deliberately not trusted with facts. A beginner tool that states a wrong distance with confidence teaches a wrong fact. Grounding the model in retrieved catalogue rows, and keeping a complete no-model path, trades a little conversational freedom for answers that are always checkable.

4. Data

Source: ESA Gaia DR3, retrieved from the public Gaia TAP service (https://gea.esac.esa.int/tap-server/tap/sync) with this ADQL query:

```
SELECT TOP 2000 source_id, ra, dec, parallax, parallax_over_error,
phot_g_mean_mag, bp_rp, teff_gspphot FROM gaiadr3.gaia_source
WHERE parallax >= 66 AND parallax_over_error > 10
ORDER BY parallax DESC
```

The parallax cut (66 mas or more) selects stars within about 15 parsecs; the quality cut leaves 1,113 stars, out to 49.4 light years. `src/prepare_data.py` computes distance from parallax and attaches common names to 14 well-known stars. A name is attached only if a Gaia source lies within 0.15 degrees of the star's published J2000 position and its Gaia distance is within 15 percent of the published distance. Eleven candidates failed these checks and were dropped. Most are the very brightest stars (Alpha Centauri A and B, Procyon, Vega, Altair, Pollux, Arcturus, Capella, Fomalhaut, Denebola), a known Gaia bright-star limitation. Seven of them are answered from a small, clearly labelled reference table of textbook (Hipparcos) distances, and the answer says plainly that the star is not in the Gaia data.

Nothing in the dataset is personal or non-redistributable; it is published survey data. The raw TAP result is kept next to the prepared file so the preparation step can be re-run and audited. Total data size is about 208 KB against a 100 MB limit. Licence: the derived dataset is released under CDLA-Permissive-2.0; the underlying measurements are ESA Gaia DR3 (citation in the References).

5. Evaluation

Evaluation here is factual correctness against the dataset itself, plus behaviour when the model is absent or failing. The app was tested in three modes natively and in Docker (see Reproducibility): fallback mode with no environment variables, LLM mode against a local mock OpenAI-compatible endpoint, and LLM mode pointed at a dead port.

Sample results, fallback mode (answers shortened only where marked):

Question	Answer (source: data-fallback)	Check
What is the closest star to us?	Proxima Centauri, 4.2 ly; Barnard's Star, 6.0 ly; Wolf 359, 7.9 ly (with magnitudes and temperatures)	Matches the three smallest distances in the CSV
How far is Tau Ceti?	"Tau Ceti is 11.9 light years away; apparent magnitude 3.3 in the Gaia G band."	CSV row: 11.91 ly
How far is Vega?	Explains Vega is too bright for Gaia in this extract and is not in the dataset; textbook distance about 25 ly	Honest non-coverage, clearly labelled
How many stars are within 10 light years?	"There are 8 stars in this Gaia DR3 dataset within 10 light years of the Sun."	Direct count over the CSV: 8
Which nearby stars are brightest?	Top 5 by Gaia G magnitude, plus a warning that Gaia's Sirius value (8.5) is saturated and far too faint	Magnitudes from the CSV; the warning is the honest part

Setup	Test	Result
No LLM configured	5 sample questions	All answered from data, 0 invented facts
Mock Apertus endpoint	"How far is Tau Ceti?"	Endpoint received the model name and the retrieved facts; response returned with source `apertus`
Dead LLM endpoint	Same question	App returned the data answer with source `data-fallback`
Docker, clean checkout	`make run`, then `/health` and `/ask`	Container served both endpoints (re-verified for this report; see BUILDLOG)

There is no separate baseline chatbot to beat; the comparison that matters is internal. The fallback and the model receive identical facts, so any factual drift in model mode is visible by diffing the answer against the `facts` field the API returns.

6. Limitations

- **Question coverage is narrow by design.** Five intent families and named-star lookup. Free-form questions outside them get the help answer, not a guess.
- **Bright stars are missing or wrong in Gaia.** Seven famous stars are reference-table answers, not Gaia measurements, and Gaia's brightness for Sirius is saturated (8.5 instead of about -1.5), so the "brightest" list excludes the actual brightest star and says so.
- **The model was tested through a mock endpoint**, not against a live hosted Apertus deployment, during this build. The interface is the standard chat-completions shape and the fallback covers endpoint failure, but phrasing quality with the real model is unmeasured.
- **Temperatures exist for only 518 of 1,113 stars** (GSP-Phot coverage), so "hottest" answers cover that subset.

- Distances use the naive 1/parallax inversion, standard for such nearby, high signal-to-noise parallaxes (all rows have parallax_over_error above 10), but it is still an approximation a research tool would not stop at.

7. Reproducibility

- `make run` from a clean checkout builds and starts everything; no accounts, keys or downloads are needed for the fallback path.
- Data preparation is deterministic: `python3 src/prepare_data.py` regenerates `nearby_stars.csv` from the bundled raw TAP result and prints every name match and drop.
- Docker verification was run on a separate machine (an iMac running Docker) because the build VM has no Docker: fresh copy of the final repository, `make run` at the repository root, then `/health` and `/ask` exercised against the running container. Details in `BUILDLOG.md` at the repository root.
- Code: Apache License 2.0. This report and the documentation: CC BY 4.0.

8. Next steps

With another month: widen the extract (more stars, proper motions so "how fast is it moving?" works), add a sky-position view ("what is near the Sun in the direction of Orion?"), evaluate Apertus phrasing against the fallback on a fixed question set with human raters, and publish the prepared dataset on Hugging Face with a dataset card.

References

- Gaia Collaboration et al. (2023), "Gaia Data Release 3: Summary of the content and survey properties", Astronomy & Astrophysics 674, A1. Data: ESA Gaia Archive, https://gea.esac.esa.int/archive/
- This work has made use of data from the European Space Agency (ESA) mission Gaia (https://www.cosmos.esa.int/gaia), processed by the Gaia Data Processing and Analysis Consortium (DPAC, https://www.cosmos.esa.int/web/gaia/dpac/consortium). Funding for the DPAC has been provided by national institutions, in particular the institutions participating in the Gaia Multilateral Agreement.
- Apertus models: https://huggingface.co/swiss-ai (Apertus-v1.5-8B and Apertus-v1.5-70B).
- Hack Apertus project template: https://github.com/HackApertus/project-template